

✦ SLEEPSCRIBE

Interview Prep

Full-Stack AI Sleep Journal - the complete project brief

Tech stack - features and the tech behind each - deep technical talking points,
and 50+ expected questions with model answers.

React - Express - MongoDB - Groq LLM - Vercel

1. Know your project (the pitch)

30-second pitch

"SleepScribe is a full-stack AI sleep journal. People record dreams by text or voice, and a large language model decodes each one into a clear interpretation - summary, mood, symbols, themes and a sleep tip. Over time it surfaces patterns: streaks, recurring symbols, how sleep length relates to dream tone, and a weekly AI sleep-coach report. It is a React front end with an Express + MongoDB backend, deployed on Vercel, with Groq running the AI."

2-minute version - add

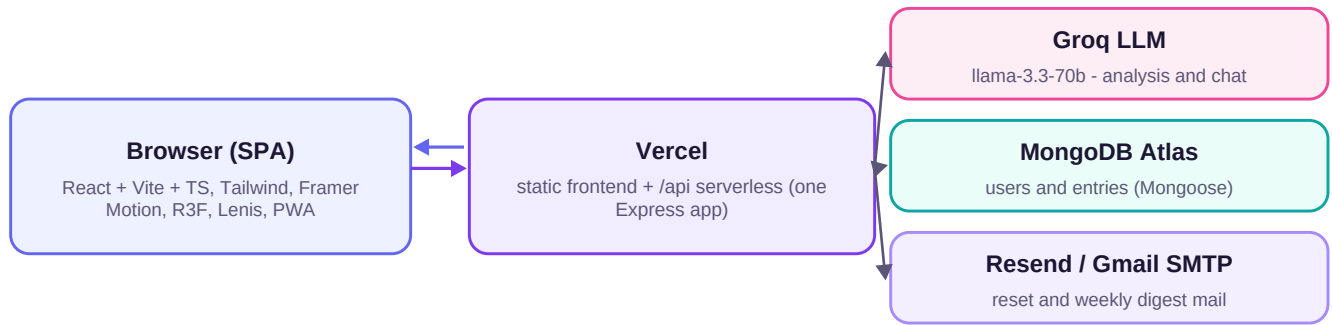
- **Two surfaces:** a polished public marketing site (anyone can try the dream AI and chat with the assistant) and a private, login-protected app where each user keeps their journal.
- **AI everywhere:** dream analysis, a journal-aware assistant (Luna), per-dream follow-up chat, and a weekly digest - all on one Groq model with a safe offline fallback.
- **Engineering highlights:** a frame-scrubbed cinematic hero, a generative-art dream visualizer, real analytics, and a fully mobile-responsive, installable PWA.

2. Tech stack (and why)

Layer	Technology	Why this choice
UI framework	React 18 + Vite + TypeScript	Component model, instant HMR, type safety
Styling	Tailwind CSS 3	Utility-first - fast, consistent, no CSS sprawl
Animation	Framer Motion, Lenis	Declarative motion + weighted smooth scroll
3D / visuals	React Three Fiber, Canvas 2D	Cinematic hero, generative dream art
Routing	React Router 6	SPA routing + protected routes
Backend	Node.js + Express 4	Familiar REST API; runs as a serverless fn
Database	MongoDB Atlas + Mongoose	Flexible documents fit the AI analysis blob
AI	Groq SDK - llama-3.3-70b	Very fast inference, generous free tier
Auth	JWT + bcrypt + Google Identity	Stateless sessions + social login
Email	Resend API / Gmail SMTP (nodemailer)	Reset + weekly digest mail
Hosting	Vercel (frontend + serverless API)	One repo, git-push deploy, cron jobs

3. Architecture

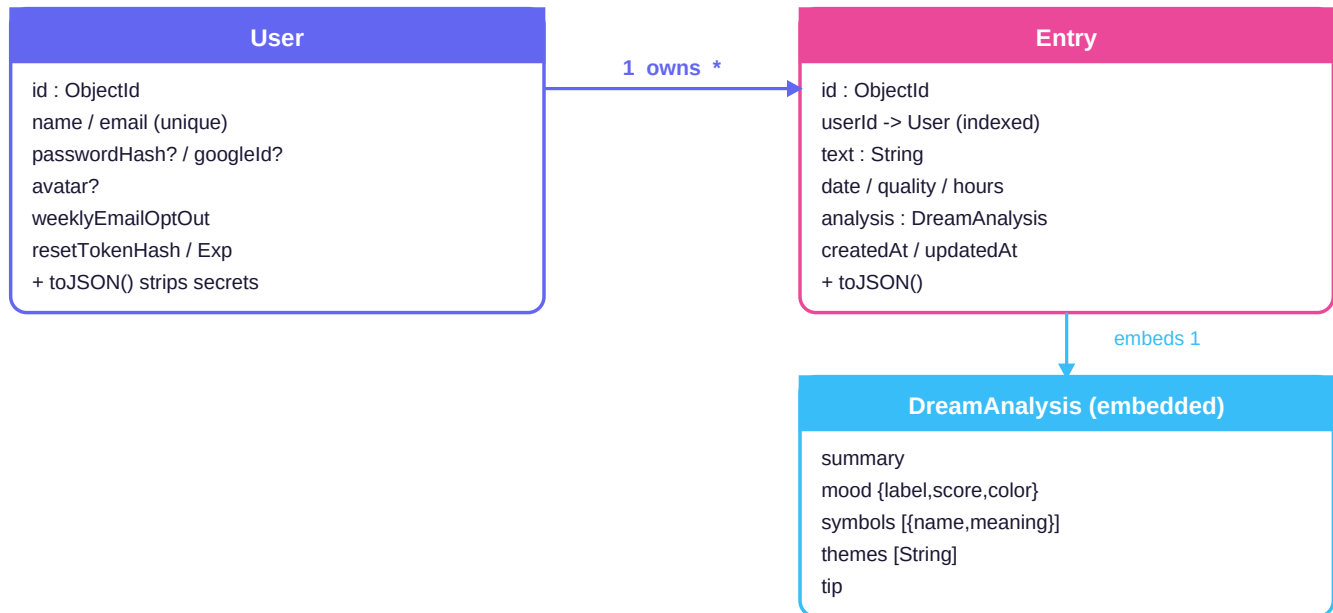
The whole app is one Vercel project: a Vite-built static front end plus a single Express app exposed as one serverless function under /api. A rewrite sends every /api/* request to that function, which does its own internal routing.



JWT auth on protected requests - API keys stay on the server

4. Data model

Two MongoDB collections. A **User** owns many **Entry** documents; each entry **embeds** the AI's **DreamAnalysis** object, so a dream and its interpretation load together with no joins.



- **Why embed the analysis?** It is 1:1 with the entry and always read together - the document model avoids a join.
- **Index:** userId on Entry, since every query filters by owner.
- **Security:** toJSON() strips the password hash and reset tokens before any response.

5. Features and the tech behind each

Feature	What it does	Tech used
Cinematic hero	240-frame video scrubs on scroll	Canvas 2D, useScroll, rAF easing
Dream AI demo	Public dream analysis, no login	Groq, per-IP rate limit, offline fallback
Luna assistant	Chatbot; journal-aware when logged in	Groq, prompt grounding on user entries
Dream journaling	Text + voice capture, AI analysis	Web Speech API, Groq, Mongoose
Per-dream chat	Follow-up Q and A about one dream	Groq multi-turn with dream context
Insights	Mood, trends, correlations, recurring	Pure TS over entries + SVG charts
Weekly digest	AI sleep-coach summary + email	Groq, Vercel Cron, Resend/Gmail
Dream art	Generative aurora image per dream	Seeded PRNG + Canvas 2D
Share card	Shareable PNG of an analysis	Canvas 2D, Web Share API
PDF export	Whole journal as a print PDF	Styled print window

Feature	What it does	Tech used
Auth	Email/password + Google + reset	JWT, bcrypt, google-auth-library
PWA	Installable, offline shell	vite-plugin-pwa / Workbox

6. Deep technical talking points

These are the 'how exactly' details an interviewer drills into.

Auth and security

- Passwords hashed with **bcrypt** (10 rounds). Login returns a signed **JWT** (HS256, payload sub = userId, 7-day expiry). The client stores it and sends Authorization: Bearer; a requireAuth middleware verifies it and attaches req.userId.
- **Google sign-in**: the front end gets a Google ID token; the server verifies it with google-auth-library against the OAuth client ID, then links or creates the user.
- **Password reset**: a random token is emailed; only its **SHA-256 hash** + 1-hour expiry are stored. The endpoint always returns the same response so attackers cannot enumerate which emails exist.
- **Secrets**: all keys are env vars; .env is git-ignored; the Groq/LLM key never reaches the browser - all AI calls are proxied through the backend.

AI integration

- Analysis uses Groq chat completions with response_format: json_object and a strict system prompt; a coerce() function validates and clamps the shape (mood 0-100, hex colour, 2-4 symbols) so a bad model response can never break the UI.
- If no key is set or the call fails, a deterministic **offline heuristic** analyzer runs - the app always returns something.
- **Journal-aware Luna** is RAG-lite: the server compresses the user's recent entries into a compact digest and injects it as system context, with rules to ground answers and never invent dreams. At this scale no vector DB is needed.
- Latency/cost controlled with max_tokens, trimmed history, and temperature per task.

Frontend and the scroll engineering

- App-wide entries live in a tiny **pub/sub store** (no Redux) so Dashboard, Journal and Insights stay in sync; writes are optimistic.
- The hero pre-decodes 240 WebP frames and draws the scroll-mapped frame on a canvas, easing toward the target each requestAnimationFrame tick; device-pixel-ratio is capped at 1.5 for cheaper draws.
- **Lenis** gives weighted smooth scrolling; data-lenis-prevent on the chat list stops Lenis hijacking the wheel so Luna scrolls natively.
- **Mobile bug fixed**: transient horizontal overflow widened the initial containing block, so fixed background layers spilled past the viewport. Fix: overflow-x: clip on html/body (clip, not hidden, so it never creates a scroll container that would break the sticky hero).

Backend, database and ops

- One Express app is exported as a Vercel function; the Mongo connection is **cached across warm invocations** (the right serverless pattern). Public AI routes are allowed to run even if the DB is down.
- A global error handler always returns JSON so the function never crashes; CORS uses an origin allow-list.
- Public AI endpoints are **rate-limited per IP** with input caps. (Honest caveat: the limiter is per warm instance - Redis/Upstash would make it global.)
- The weekly digest runs on **Vercel Cron**, secured by a CRON_SECRET bearer token so it cannot be triggered publicly.

7. Expected interview questions (with answers)

Project and behavioural

Q. Walk me through this project.

A. SleepScribe is a full-stack AI sleep journal. Users log dreams by text or voice; a Groq LLM decodes each into a summary, mood, symbols, themes and a tip. It adds streaks, recurring-symbol detection, sleep-vs-dream correlations and a weekly AI digest. React + Vite + TS front end, Express + MongoDB backend, deployed on Vercel as a static site plus one serverless API.

Q. What problem does it solve?

A. Dream journaling helps self-reflection but is tedious and the meaning is opaque. SleepScribe makes capture frictionless (voice + text) and instantly turns raw entries into structured, personal insight, building a habit with streaks and weekly reports.

Q. What was the hardest part?

A. Two things: making the cinematic, animation-heavy site fully responsive - I traced a horizontal-overflow bug that widened the containing block and fixed it with `overflow-x: clip` - and grounding the AI in each user's own journal without standing up a vector database.

Q. What are you most proud of?

A. The journal-aware assistant: when you are signed in, Luna answers from your own dream history (recurring symbols, mood shifts), which makes it feel personal rather than generic.

Q. If you had more time, what would you improve?

A. Collect real sleep hours/quality in the journal form (today they are placeholder values), move rate limiting to Redis, add embeddings for journal search at scale, and verify an email domain so digests can reach all users.

Frontend

Q. Why React + Vite + TypeScript?

A. React for a composable UI, Vite for near-instant HMR and fast builds, TypeScript to catch shape errors at compile time - important when passing the AI's analysis object around.

Q. How do you share state across pages without Redux?

A. A lightweight module-level store with a pub/sub pattern (a Set of listeners + an in-memory cache). Components subscribe via a hook; any add/remove emits and re-renders all subscribers. Redux would be overkill for this scope.

Q. How does the scroll animation actually work?

A. A 261/240-frame WebP sequence is preloaded and decoded up front. Framer Motion's `useScroll` gives a 0-1 progress for the section; each rAF tick I ease the current frame index toward progress and draw that frame on a canvas with a cover fit at capped DPR. Easing gives it weight and avoids jumps.

Q. What is Lenis and why data-lenis-prevent?

A. Lenis is a smooth-scroll library that intercepts wheel/touch to animate the page. It was stealing the gesture over the chat box, so the inner list would not scroll. Adding `data-lenis-prevent` tells Lenis to ignore that element so native overflow scrolling works.

Q. Tell me about a CSS/layout bug you fixed.

A. On mobile the fixed background layers spilled past the viewport. `window.innerWidth` read wider than the layout because transient horizontal overflow widened the initial containing block. I added `overflow-x: clip` on `html` and `body` - `clip` rather than `hidden`, because `hidden` would create a scroll container and break the sticky hero and Lenis.

Q. How do you handle accessibility and reduced motion?

A. Semantic elements and aria-labels on icon buttons, and a global prefers-reduced-motion media query that disables animations; the hero also renders a static final frame for those users.

Backend and API

Q. How does an Express app run on Vercel serverless?

A. The Express app is built with no app.listen and exported; a thin api/index.js re-exports it as the function handler. vercel.json rewrites every /api/* path to that one function, which routes internally. So one function serves the whole API.

Q. How do you manage the DB connection in serverless?

A. connectDB() caches the Mongoose connection on the global object, so a warm invocation reuses it instead of reconnecting per request - the standard serverless pattern. A middleware ensures it is connected before DB-backed routes.

Q. How do you protect public AI endpoints from abuse?

A. Per-IP rate limiting (a sliding window in a Map) plus input length caps and history trimming. I am explicit that this is per warm instance, so for production I would move it to Redis/Upstash for a global limit.

Q. How is error handling done?

A. A global Express error middleware logs and returns a JSON error with the right status, so the function never crashes and the client always gets parseable JSON. Public AI routes also fall back to canned/heuristic responses.

Q. How would you design the REST API?

A. Resource-oriented: /api/auth/*, /api/entries (GET/POST/DELETE), /api/ai/* for analyze/chat/assistant/journal/digest, and /api/cron/* for scheduled jobs. Auth via Bearer JWT; consistent JSON error shape.

Database

Q. Why MongoDB over SQL?

A. The AI analysis is a nested, evolving blob (mood, arrays of symbols/themes) that maps naturally to a document and changes shape over time. The access pattern is simple owner-scoped reads, so I do not need relational joins.

Q. How did you model the data?

A. User and Entry collections. Entry stores userId (indexed) and embeds the DreamAnalysis sub-document. Embedding is right here because analysis is 1:1 with the entry and always read together.

Q. How do you stop sensitive fields leaking?

A. Each model overrides toJSON() to return only safe fields - the password hash and reset tokens are never serialized to the client.

Q. What indexes and why?

A. userId on Entry (every list/query filters by user), and a unique index on User.email. I would add a compound index on (userId, createdAt) as the journal grows since it sorts by date.

AI / LLM

Q. How do you get reliable structured output from the model?

A. I request JSON mode (response_format json_object) with an exact schema in the system prompt, then validate and clamp it server-side with a coerce() function. Anything malformed falls back to defaults, so the UI is never broken by the model.

Q. Why Groq and that model?

A. Groq's inference is extremely fast (good for an interactive chat/analysis UX) with a generous free tier, running llama-3.3-70b which is strong enough for interpretation and conversation.

Q. Is the journal-aware assistant RAG? How does it work?

A. It is retrieval-augmented in spirit: the server pulls the user's recent entries, compresses them into a compact digest, and injects it as system context with rules to ground answers and not invent dreams. At this scale that beats a vector DB; to scale I would embed entries and retrieve top-k.

Q. How do you handle hallucination and cost?

A. Grounding plus an explicit instruction to say when the journal does not contain the answer; cost/latency via max_tokens, trimming history to the last N turns, and task-specific temperature.

Q. What if the AI is down or has no key?

A. Analysis falls back to a deterministic offline heuristic analyzer, and chat returns a friendly canned reply - the app degrades gracefully instead of erroring.

Security

Q. How is authentication implemented?

A. Stateless JWT: bcrypt-verified login issues an HS256 token (sub=userId, 7d). Protected routes use a requireAuth middleware that verifies the Bearer token and sets req.userId. No server session store needed.

Q. How is password reset kept secure?

A. A random token is emailed but only its SHA-256 hash and a 1-hour expiry are stored; the request endpoint returns an identical response whether or not the email exists, preventing user enumeration.

Q. Where do API keys live and how are they protected?

A. In environment variables, never in the repo (.env is git-ignored) and never shipped to the browser - the front end calls our backend, which calls Groq. CORS is restricted to an allow-list of origins.

Q. What other risks did you consider?

A. Input caps to limit payload/token abuse, rate limiting on open endpoints, React's automatic escaping against XSS, and least-privilege API keys (the email key is send-only).

Deployment, DevOps and scaling

Q. How is it deployed and what is the CI/CD?

A. Vercel builds the Vite front end and the serverless API from the same repo. Pushing to main triggers an automatic production deploy. Env vars are set in the Vercel dashboard per environment.

Q. How do scheduled jobs work?

A. Vercel Cron hits /api/cron/weekly-digest on a schedule; the endpoint checks a CRON_SECRET bearer token so it cannot be triggered publicly, then emails each active user their weekly digest.

Q. How would you scale this to a million users?

A. Add proper indexes and pagination; move rate limiting and caching to Redis; queue outbound email and use a verified domain; introduce embeddings + a vector store for journal chat; and rely on Vercel's serverless autoscaling with Atlas scaling/sharding behind it.

Q. Your rate limiter is per warm instance - is that a problem?

A. Yes, across many instances the global limit is looser than intended. It is a guardrail, not a guarantee. The fix is a shared store (Upstash Redis) keyed by IP so all instances share one counter.

Tradeoffs and gotchas

Q. Why not Next.js?

A. The app is a client-heavy SPA with a separate API; Vite + a single Express function kept the mental model simple and the build fast. Next would help if I needed SSR/SEO on authenticated pages, which I did not.

Q. overflow-x: clip vs hidden - why clip?

A. hidden makes the other axis compute to auto, creating a scroll container that breaks position: sticky and Lenis. clip contains overflow without establishing a scroll container, so the sticky hero and smooth scroll keep working.

Q. Why did the flex chat list not scroll, and the fix?

A. A flex child defaults to min-height: auto, so it grows to fit content instead of scrolling. Adding min-h-0 lets it shrink within the fixed-height card so overflow-y: auto actually scrolls.

Q. What is a known limitation you would call out?

A. Sleep hours and quality are currently generated at entry creation rather than entered by the user, so those specific metrics are illustrative. The analytics math is real; the inputs need a form field, which is my next step.

8. One-line answers to keep ready

If asked...	Say
Stack in one breath	React/Vite/TS + Tailwind, Express + MongoDB, Groq AI, on Vercel.
Why serverless Express	One app, git-push deploy, autoscale, cron - no server to run.
Auth	Stateless JWT + bcrypt, plus Google sign-in.
AI reliability	JSON mode + server-side validation + offline fallback.
Biggest bug	Mobile overflow widening the containing block - fixed with overflow-x: clip.
Next improvement	Real sleep inputs, Redis rate limiting, embeddings for journal search.

◆ **Tip: for any feature, be ready to say WHAT it does, the TECH behind it, and one TRADEOFF you made.**